Database Management Systems Project Report

Griffin Shea

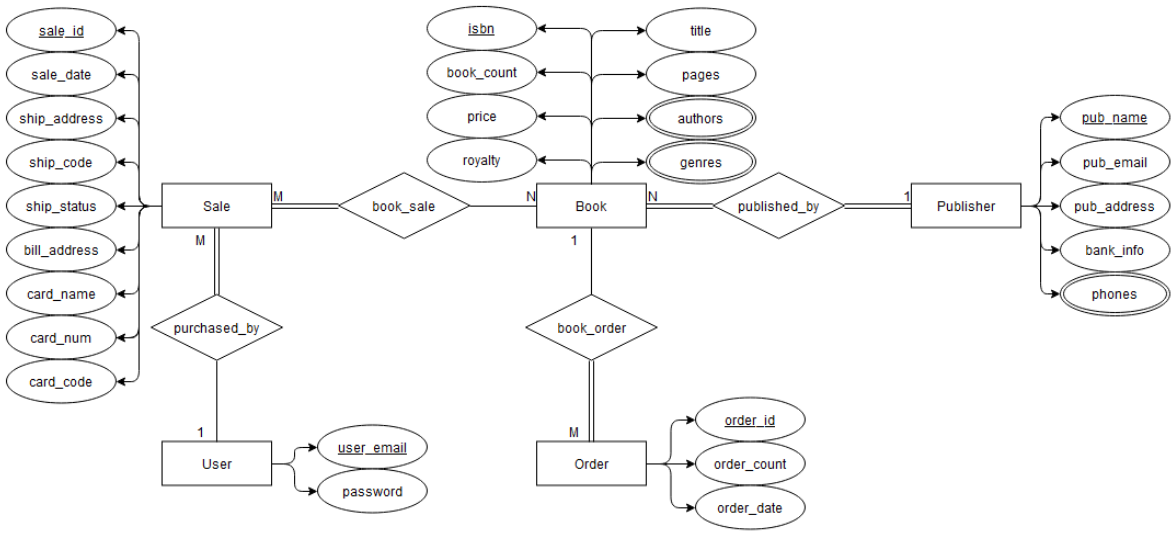
I. Conceptual Design
II. Reduction to Relational Schemas
III. Normalization of Relational Schemas
IV. Database Schema Diagram
V. Problem Statement Breakdown and User Stories

I. Conceptual Design

Section **V.** includes user stories describing how the system will work. These are some assumptions I have made to produce the ER diagram. It is followed by the ER diagram itself and then a list describing each entity attribute and each relationship:

Assumptions:

- Publishers are not added to the system until a Book associated with that publisher has been added.
- Books with an anonymous author are associated with an Author with name "anonymous" (it is not NULL).
- A new user is created with email and password populated after registration confirmation.
- Users may only order one copy per book per purchase.
- All publisher attributes must be unique (no shared emails, addresses, phone numbers, etc.).
- Publishers are payed only in royalties, books are sent to the store without any upfront cost.
- For the purposes of functional dependencies later on, we also assume each shipping code from the third-party delivery service is unique



Sale:

- sale_id: unique identifier for each book purchase.
- sale_date: date of purchase.
- ship_address: shipping address.
- ship_code: shipping code from third-party shipping service.
- ship_status: current status of shipment from third-party shipping service, pulled from third-party website using the shipping code for customer convenience.
- bill_address: billing address.
- card_name: name on credit card.
- card_num: 16-digit credit card number.
- card_code: the three wacky numbers on the back.

Order:

- order_id: unique order identifier.
- order_count: number of books ordered.
- order_date: date email was sent to publisher

Book:

- isbn: a unique identifier for books used internationally.
- book_count: how many copies currently available in the warehouse.
- price: price of the book user when purchased by a customer.
- royalty: percentage of purchase price sent to publisher.
- title: title of the book.
- pages: number of pages in the book.
- authors: list of authors involved in writing the book.
- genres: list of genres the book has been tagged as (to help users search).

User:

- user_email: user's email used to login and identify their account.
- password: user's password used to login.

Publisher:

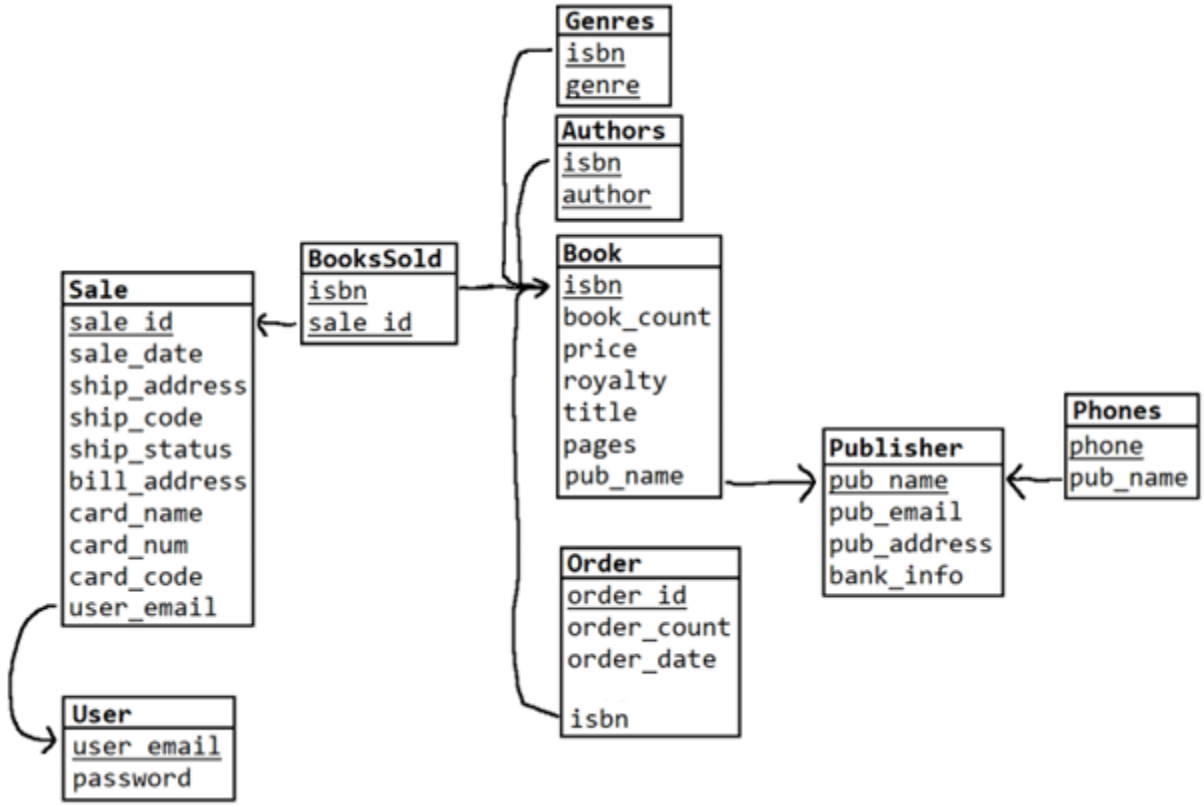
- pub_name: name of publisher.
- pub_email: email of publisher.
- pub_address: mailing address of publisher.
- bank_info: publisher deposit account.
- phones: list of publisher phone numbers.

purchased_by → Associates each and every sale/purchase with the (individual) user who made the purchase. One user may have multiple orders.
book_sale → One sale/purchase may have multiple books (only one copy of each individual book per purchase) associated with it, and of course, the same book can be purchased multiple times.
book_order → One book can be ordered multiple times, each order regards to one book and one book only.

published_by → Each book has one associated publisher, and a publisher can have multiple books in the store (but never zero, because publishers are only added to the system when the store carries their books for the first time).

II. Reduction to Relation Schemas

The only non-trivial mappings are the attributes genres, authors, and phones, which have become relations, and the book_sale relationship has also become a relation.



III. Normalization of Relation Schemas

The following table lists each functional dependency by the relational schema in which it is not trivial. No functional dependencies apply to BooksSold, Genres, or Authors. All relations are in Boyce-Codd normal form, the third column in the table sketches out the proof showing that the left operand of the non-trivial functional dependency is a superkey. Keep in mind the following definitions and rules:

BCNF:

$$Relational\ schema\ R\ is\ in\ BCNF \Leftrightarrow for\ each\ functional\ dependency\ \alpha \rightarrow \beta:$$

$$(\alpha \rightarrow \beta\ is\ trivial\ for\ R) \vee (\alpha \rightarrow \beta\ is\ a\ superkey\ for\ R)$$

Superkey:

$$Functional\ dependency\ \alpha \rightarrow \beta\ is\ a\ superkey\ for\ relational\ schema\ R \Leftrightarrow (\alpha \rightarrow R) \wedge (\alpha \subseteq R)$$

Reflexivity:

$$\alpha \subseteq \beta \Rightarrow \alpha \rightarrow \beta$$

Union:

$$\alpha \rightarrow \beta \wedge \alpha \rightarrow \gamma \Rightarrow \alpha \rightarrow \beta\gamma$$

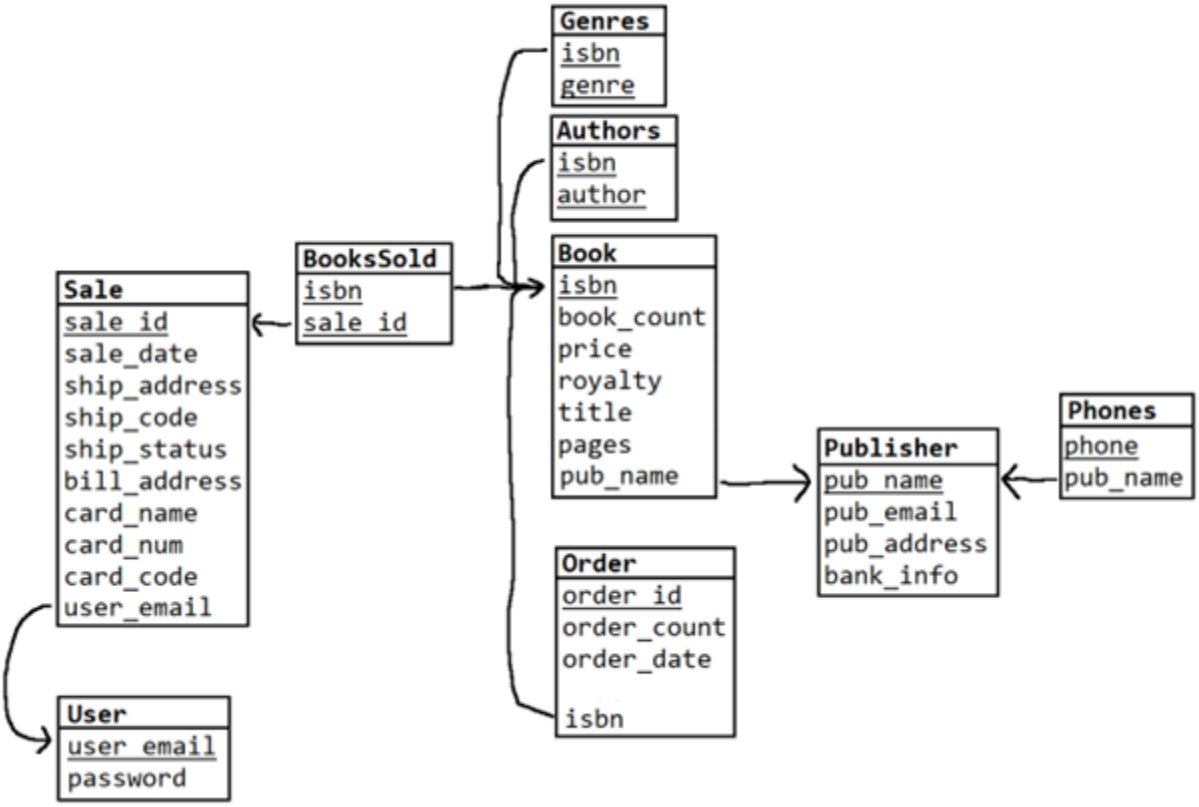
SCHEMA	FUNCTIONAL DEPENDENCY	BCNF PROOF
User	user_email → password	1. R = {user_email, password} 2. user_email → password 3. user_email ⊆ R (via 1) 4. user_email ⊆ user_email 5. user_email → user_email (reflexivity, 4) 6. user_email → user_email, password (union, 2, 5) 7. user_email → R (via 1, 6) 8. user_email is superkey for relation (definition of superkey, 3, 7)
Sale	sale_id → sale_date, ship_address, ship_code, ship_status, bill_address, card_name, card_num, card_code, user_email	1. R = {sale_id, sale_date, ship_address, ship_code, ship_status, bill_address, card_name, card_num, card_code, user_email} 2. sale_id → sale_date, ship_address, ship_code, ship_status, bill_address, card_name, card_num, card_code, user_email 3. sale_id ⊆ R (via 1) 4. sale_id ⊆ sale_id 5. sale_id → sale_id (reflexivity, 4)

		6. sale_id → sale_id, sale_date, ship_address, ship_code, ship_status, bill_address, card_name, card_num, card_code, user_email (union, 2, 5) 7. sale_id → R (via 1, 6) 8. sale_id is superkey for relation (definition of superkey, 3, 7)
	ship_code → sale_id, sale_date, ship_address, ship_status, bill_address, card_name, card_num, card_code, user_email	1. R = {sale_id, sale_date, ship_address, ship_code, ship_status, bill_address, card_name, card_num, card_code, user_email} 2. ship_code → sale_id, sale_date, ship_address, ship_status, bill_address, card_name, card_num, card_code, user_email 3. ship_code ⊆ R (via 1) 4. ship_code ⊆ ship_code 5. ship_code → ship_code (reflexivity, 4) 6. ship_code → sale_id, sale_date, ship_address, ship_code, ship_status, bill_address, card_name, card_num, card_code, user_email (union, 2, 5) 7. ship_code → R (via 1, 6) 8. ship_code is superkey for relation (definition of superkey, 3, 7)
Book	isbn → book_count, price, royalty, title, pages, pub_name	1. R = {isbn, book_count, price, royalty, title, pages, pub_name} 2. isbn → book_count, price, royalty, title, pages, pub_name 3. isbn ⊆ R (via 1) 4. isbn ⊆ isbn 5. isbn → isbn (reflexivity, 4) 6. isbn → isbn, book_count, price, royalty, title, pages, pub_name (union, 2, 5) 7. isbn → R (via 1, 6) 8. isbn is superkey for relation (definition of superkey, 3, 7)
Order	order_id → order_count, order_date, isbn	1. R = {order_id, order_count, order_date, isbn} 2. order_id → order_count, order_date, isbn 3. order_id ⊆ R (via 1) 4. order_id ⊆ order_id 5. order_id → order_id (reflexivity, 4) 6. order_id → order_id, order_count, order_date, isbn (union, 2, 5) 7. order_id → R (via 1, 6) 8. order_id is superkey for relation (definition of superkey, 3, 7)
Publisher	pub_name → pub_email, pub_address, bank_info	1. R = {pub_name, pub_email, pub_address, bank_info } 2. pub_name → pub_email, pub_address, bank_info 3. pub_name ⊆ R (via 1)

		4. pub_name ⊆ pub_name 5. pub_name → pub_name (reflexivity, 4) 6. pub_name → pub_name, pub_email, pub_address, bank_info (union, 2, 5) 7. pub_name → R (via 1, 6) 8. pub_name is superkey for relation (definition of superkey, 3, 7)
	pub_email → pub_name, pub_address, bank_info	1. R = {pub_name, pub_email, pub_address, bank_info } 2. pub_email → pub_name, pub_address, bank_info 3. pub_email ⊆ R (via 1) 4. pub_email ⊆ pub_email 5. pub_email → pub_email (reflexivity, 4) 6. pub_email → pub_name, pub_email, pub_address, bank_info (union, 2, 5) 7. pub_email → R (via 1, 6) 8. pub_email is superkey for relation (definition of superkey, 3, 7)
	pub_address → pub_name, pub_email, bank_info	1. R = {pub_name, pub_email, pub_address, bank_info } 2. pub_address → pub_name, pub_email, bank_info 3. pub_address ⊆ R (via 1) 4. pub_address ⊆ pub_address 5. pub_address → pub_address (reflexivity, 4) 6. pub_address → pub_name, pub_email, pub_address, bank_info (union, 2, 5) 7. pub_address → R (via 1, 6) 8. pub_address is superkey for relation (definition of superkey, 3, 7)
	bank_info → pub_name, pub_email, pub_address	1. R = {pub_name, pub_email, pub_address, bank_info } 2. bank_info → pub_name, pub_email, pub_address 3. bank_info ⊆ R (via 1) 4. bank_info ⊆ bank_info 5. bank_info → bank_info (reflexivity, 4) 6. bank_info → pub_name, pub_email, pub_address, bank_info (union, 2, 5) 7. bank_info → R (via 1, 6) 8. bank_info is superkey for relation (definition of superkey, 3, 7)
Phones	phone → pub_name	1. R = {phone, pub_name} 2. phone → pub_name 3. phone ⊆ R (via 1) 4. phone ⊆ phone 5. phone → phone (reflexivity, 4) 6. phone → phone, pub_name (union, 2, 5) 7. phone → R (via 1, 6) 8. phone is superkey for relation (definition of superkey, 3, 7)

IV. Database Schema Diagram

No changes needed to be made as the relation schemas in the previous diagram were already in BCNF as shown in the table above.



V. Problem Statement Breakdown and User Stories

Customer user stories:

1. User enters keyword into the search box and selects a feature from (title, author, isbn, genre, publisher):
   - 1.1. System displays a list of books searched with the keyword-feature pair.
   - 1.2. User may:
     - 1.2.1. Selects a book in the list or
     - 1.2.2. return to search screen (go to 1).
   - 1.3. System displays information about selected book (authors, genres, publisher, page count, price, count in warehouse).
   - 1.4. User may:
     - 1.4.1. Add book to checkout basket and return to list screen or
     - 1.4.2. return to list screen (go to 1.2) or
     - 1.4.3. return to search screen (go to 1).

2. User selects to go to checkout:
   - 2.1. If user is not logged in:
     - 2.1.1. System prompts user to log in with email and password.
     - 2.1.2. User submits email and password.
     - 2.1.3. If email and password do not match an account in the system:
       - 2.1.3.1. System displays login error and returns to prompt screen (go to 2.1.1).
       - 2.1.3.2. Otherwise, continue (go to 2.2).
   - 2.2. System displays books user has added to cart in this session and calculates a purchase price based on price of each book.
   - 2.3. User inputs billing and shipping information and completes order.
   - 2.4. System assigns a unique sale ID to this purchase and displays for the user.
   - 2.5. System sends royalties to publishers of books included in the sale.
   - 2.6. System removes copies of each book from warehouse count and authorizes an agent to collect those books and deliver them through third-party shipping company
     - 2.6.1. When the shipment is confirmed through third-party shipping company, agent will send shipping code to user through email.
   - 2.7. System checks current book counts against book order limit for each book sold and if the limit is broken:
     - 2.7.1. Email publisher to order new copies (count sales associated with that book dated to the previous month).

3. User inputs shipping code or sale ID.
   - 3.1. System displays shipping status of order (shipping status is pulled using shipping code on third-party shipping company API).

Owner user stories:

1. User enters a new book into the warehouse specifying isbn, title, author(s), genre(s), price, royalty, and the number of books added to the warehouse.
   1.1. User may:
      1.1.1. Select a publisher from the list of publishers already entered into the database.
      1.1.2. Enter publisher name, email, mail address, and banking information to be added to the database.
   1.2. System adds book and publisher data to database.

2. User selects to view a report:
   2.1. System uses database to generate and display report.